Práctica. Pruebas automatizadas con pytest

Nombre y apellidos: _____

Martes 3 de noviembre de 2026 · PPS · 4 horas · trabajo individual

En la práctica anterior visteis que unos tests en verde y un 94 % de cobertura no garantizan nada: la tienda seguía regalando dinero. Hoy damos un paso más. Os doy un proyecto pequeño con una especificación por escrito y vuestro trabajo es convertir esa especificación en pruebas. Si las pruebas están bien hechas, van a encontrar los fallos que he dejado escondidos, y uno de ellos es un problema de seguridad de los que se ven en empresas de verdad.

No quiero que os limitéis a copiar los ejemplos. Cada prueba que escribáis tiene que salir de una frase de la especificación, y tenéis que saber decirme de cuál.

1. Qué vas a aprender hoy

Al terminar la sesión tienes que ser capaz de:

1. Organizar un proyecto con el código en una carpeta y las pruebas en otra, y configurar pytest para que lo encuentre todo.
2. Escribir pruebas a partir de una especificación, sin mirar primero el código.
3. Usar `@pytest.mark.parametrize` para probar muchos casos con una sola función.
4. Usar las fixtures `tmp_path` y una fixture propia en `conftest.py`.
5. Comprobar excepciones con `pytest.raises` y el parámetro `match`.
6. Leer el informe de un test que falla y saber qué te está diciendo.
7. Medir la cobertura con `pytest-cov` y hacer que la ejecución falle si baja de un mínimo.

2. Lo que necesitas

Conocimientos: lo que hicimos en la práctica 2 (entorno virtual, `pytest`, `coverage`, test de regresión). Si no la terminaste, empieza por el itinerario de recuperación del apartado 9.

Equipo: tu Ubuntu del aula o tu portátil con Python 3.10 o superior. No necesitas Docker ni Git para esta práctica salvo para la entrega final.

Herramientas que vamos a usar: `python3`, `venv`, `pip`, `pytest` y `pytest-cov`.

3. Prepara el entorno (20 minutos)

Descarga el proyecto `PPS-2026-11-03-pytest-LAB.zip` de la web del curso (PPS, Prácticas) y descomprímelo en tu carpeta de trabajo:

```
mkdir -p ~/pps
cd ~/pps
unzip ~/Descargas/PPS-2026-11-03-pytest-LAB.zip
cd PPS-2026-11-03-pytest-LAB
ls
```

Tienes que ver `ESPECIFICACION.md`, `README.md`, `pyproject.toml` y las carpetas `inscripciones` y `tests`. La primera tiene el código; la segunda, las pruebas.

Crea el entorno virtual, actívalo e instala las dos herramientas:

```
python3 -m venv .venv
. .venv/bin/activate
pip install pytest pytest-cov
```

La primera orden crea una carpeta `.venv` con un Python solo para este proyecto. La segunda la activa: a partir de ahí, `python` y `pip` son los del entorno y verás `(.venv)` al principio del prompt. La tercera instala pytest y su complemento de cobertura dentro del entorno, sin tocar el sistema.

Comprueba que todo está en su sitio:

```
python -m pytest --version
```

Tiene que responder con algo como `pytest 9.x.x` (el número exacto puede variar). Si te sale un error, mira el apartado 8 antes de seguir.

4. Lee la especificación antes que el código (20 minutos)

Abre `ESPECIFICACION.md` y léelo entero. Tiene cinco reglas, de la R1 a la R5. Son las que tiene que cumplir el código, ni más ni menos.

Ahora, **sin abrir todavía la carpeta `inscripciones`**, escribe en tu cuaderno una lista de casos que probarías para cada regla. Por ejemplo, para la R4 no basta con "un adulto paga 60". La regla dice "de 16 a 25 ambos incluidos", así que hay que probar justo 16, justo 25 y justo 26. Los fallos se esconden en los límites.

Esto es lo que hace un equipo de calidad en una empresa: las pruebas salen de lo que el cliente ha pedido, no de lo que el programador ha escrito. Si lees primero el código, acabas probando lo que hace y no lo que debería hacer.

5. Ejecuta las pruebas que ya hay (15 minutos)

```
python -m pytest
```

pytest busca las pruebas en la carpeta `tests` porque se lo indica `pyproject.toml`, y encuentra el código porque en ese mismo fichero está `pythonpath = ["."]`. Tienes que ver algo parecido a esto:

```
configfile: pyproject.toml
testpaths: tests
collected 5 items

tests/test_almacen.py .                                  [ 20%]
tests/test_validacion.py ....                            [100%]

========================= 5 passed in 0.01s =========================
```

Todo verde. Ahora mide la cobertura:

```
python -m pytest --cov=inscripciones --cov-report=term-missing
```

`--cov=inscripciones` dice qué paquete medir y `--cov-report=term-missing` añade una columna con las líneas que ninguna prueba ha ejecutado. Apunta el total que te sale y qué líneas aparecen en `Missing`. Te adelanto que la cobertura no es mala, y sin embargo el código tiene varios fallos. Vamos a por ellos.

6. Escribe las pruebas de la especificación (100 minutos)

Crea un fichero nuevo, `tests/test_especificacion.py`. Ahí vas a ir metiendo tus pruebas, regla por regla. Ejecuta pytest cada vez que añadas una o dos; no esperes a tenerlas todas.

6.1. Muchos casos con una sola función: parametrize

Para la R1 (DNI) tienes que probar muchos valores. En vez de escribir diez funciones casi iguales, usa `parametrize`:

```
import pytest
from inscripciones.validacion import validar_dni


@pytest.mark.parametrize("dni, esperado", [
    ("12345678Z", True),
    ("12345678A", False),
])
def test_r1_dni(dni, esperado):
    assert validar_dni(dni) is esperado
```

pytest ejecuta la función una vez por cada pareja de la lista y las cuenta como pruebas distintas. Si una falla, el informe te dice exactamente con qué valores: `test_r1_dni[12345678A-False]`.

Completa la lista con los casos que apuntaste en el apartado 4. Tienes que cubrir, como mínimo, la letra en minúscula, los espacios al principio y al final, un número con menos de 8 cifras y otro con más. Los DNI de la práctica son inventados: calcula tú la letra correcta con la tabla de la especificación. Por ejemplo, `00000001` da resto 1, que corresponde a la `R`.

6.2. Comprobar que algo falla: pytest.raises

La R3 y la R4 dicen que algunos datos tienen que lanzar `ValueError`. Eso se prueba así:

```
from inscripciones.validacion import normalizar_nombre


def test_r3_nombre_vacio():
    with pytest.raises(ValueError, match="vacío"):
        normalizar_nombre("   ")
```

La prueba pasa solo si dentro del bloque `with` se lanza un `ValueError` cuyo mensaje contiene la palabra `vacío`. Si no se lanza nada, pytest lo marca como fallo con `DID NOT RAISE`. Usa `match` siempre que puedas: así te aseguras de que el error es el que esperas y no otro distinto por casualidad.

Haz lo mismo para la edad mínima de la R4 y escribe con `parametrize` los casos de cuota: 16, 25 y 26 años, con y sin desempleo.

6.3. Ficheros sin ensuciar tu carpeta: tmp_path

La R5 habla del CSV que se guarda en disco. Para probarlo no quieres crear ficheros de verdad en tu proyecto. pytest te da una carpeta temporal nueva para cada prueba si pones `tmp_path` como parámetro:

```
from inscripciones.almacen import guardar_inscripcion, leer_inscripciones


def test_r5_formula(tmp_path):
    ruta = tmp_path / "inscripciones.csv"
    datos = {
        "dni": "12345678Z",
        "nombre": "=1+1",
        "email": "ana@ejemplo.es",
        "edad": 30,
        "cuota": 60.0,
    }
    guardar_inscripcion(ruta, datos)
    assert leer_inscripciones(ruta)[0]["nombre"] == "'=1+1"
```

Esta prueba pasa. Pero la R5 enumera seis caracteres peligrosos, no uno. Convierte esta prueba en una con `parametrize` que pruebe los seis. Para el tabulador y el retorno de carro, escribe `"\tx"` y `"\rx"`.

6.4. Datos de prueba reutilizables: una fixture propia

Habrás visto que el diccionario de la inscripción se repite. Crea el fichero `tests/conftest.py` con esto:

```
import pytest


@pytest.fixture
def inscripcion():
    return {
        "dni": "12345678Z",
        "nombre": "Ana García",
        "email": "ana@ejemplo.es",
        "edad": 30,
        "cuota": 60.0,
    }
```

Todas las pruebas de la carpeta pueden recibir `inscripcion` como parámetro, igual que reciben `tmp_path`, y cada una recibe un diccionario nuevo, así que si una lo modifica no afecta a las demás. Reescribe tus pruebas de la R5 para que la usen.

6.5. Lee bien lo que te dice pytest

Cuando una prueba falle, no corras a cambiar el código. Lee el informe. Por ejemplo, si pruebas que alguien de 25 años paga 40 €, vas a ver algo así:

```
    def test_cuota_25_anos():
>       assert calcular_cuota(25) == 40.0
E       assert 60.0 == 40.0
E        +  where 60.0 = calcular_cuota(25)

tests/test_validacion.py:21: AssertionError
```

La línea con `>` es la que ha fallado. Las líneas con `E` te dicen qué valor ha salido (60.0) y cuál esperabas (40.0). Con eso ya sabes que el fallo está en el cálculo de la cuota y en qué caso concreto. El nombre de la prueba y el número de línea serán otros en tu caso; lo que importa es leer las líneas `>` y `E`.

Tres opciones de pytest que os van a ahorrar mucho tiempo:

```
python -m pytest -k cuota   # solo las que tienen "cuota" en el nombre
python -m pytest --lf       # solo las que fallaron la última vez
python -m pytest -x         # para en el primer fallo
```

6.6. Apunta lo que encuentras

Antes de arreglar nada, apunta en tu cuaderno cada prueba que falla: qué regla incumple, con qué dato falla y qué valor devuelve el código. Necesitarás esa lista para la entrega. Te digo una cosa: hay más de un fallo y están en los dos ficheros.

7. Corrige el código y fija la cobertura (40 minutos)

Ahora sí, abre `inscripciones/validacion.py` y `inscripciones/almacen.py` y corrige los fallos uno por uno. Después de cada corrección vuelve a ejecutar:

```
python -m pytest --lf
```

Cuando no quede ninguna prueba roja, ejecuta todas otra vez para comprobar que no has roto nada que antes funcionaba.

Corrige siempre el código, nunca la prueba, salvo que la prueba contradiga la especificación. Si tienes la tentación de cambiar `40.0` por `60.0` en una prueba para que pase, para y vuelve a leer la R4.

Al final, haz que pytest exija una cobertura mínima:

```
python -m pytest --cov=inscripciones --cov-report=term-missing --cov-fail-under=90
```

Si la cobertura total baja del 90 %, la ejecución termina con error y te lo dice con el mensaje `FAIL Required test coverage of 90% not reached`. Si llega, verás `Required test coverage of 90% reached`. Un aviso: la cobertura solo te dice qué líneas se han ejecutado, no si las has comprobado bien. Ya lo visteis en la práctica anterior con el `assert True`.

8. Si algo no funciona

`error: externally-managed-environment` al hacer `pip install`: estás usando el pip del sistema. Activa el entorno con `. .venv/bin/activate` y repite.

`The virtual environment was not created successfully because ensurepip is not available`: a tu Ubuntu le falta el paquete de entornos virtuales. Instálalo con `sudo apt install python3-venv` y vuelve a crear el entorno.

`No module named pytest`: has abierto otra terminal y no has activado el entorno. Vuelve a la carpeta del proyecto y actívalo.

`ModuleNotFoundError: No module named 'inscripciones'`: estás ejecutando pytest desde otra carpeta. Tienes que estar en `PPS-2026-11-03-pytest-LAB`, donde está `pyproject.toml`. Compruébalo con `pwd` y `ls`.

`collected 0 items`: el fichero o la función no empiezan por `test_`. pytest solo recoge `test_*.py` y funciones `test_*`.

`fixture 'inscripcion' not found`: el fichero se tiene que llamar exactamente `conftest.py` y estar dentro de `tests`.

`unrecognized arguments: --cov`: no está instalado `pytest-cov` en el entorno activo. Instálalo con `pip install pytest-cov`.

Sin Internet en el aula: si `pip install` no puede descargar nada, en Ubuntu puedes instalar las mismas herramientas desde los paquetes del sistema con `sudo apt install python3-pytest`

`python3-pytest-cov` y ejecutar `python3 -m pytest` sin entorno virtual.

9. Elige tu itinerario (40 minutos)

Recuperación

Si tienes pendiente la práctica 2 (código seguro), termínala antes de seguir con esta. Lo mínimo que tienes que tener es la lectura de los tres validadores (PPS-E1) y el test de regresión de la incidencia #17 fallando primero y pasando después. Cuando la tengas, vuelve al apartado 6 de esta hoja y haz al menos las reglas R1 y R4.

Ampliación

Si has terminado, elige una de estas dos:

1. La secretaría quiere aceptar también NIE: una `X`, `Y` o `Z` seguida de 7 cifras y una letra. Para calcular la letra, la `X` vale 0, la `Y` vale 1 y la `Z` vale 2, se pone delante de las cifras y se usa la misma tabla. Escribe primero las pruebas (que fallarán) y después amplía `validar_dni`. Un NIE inventado para probar: `X0000000T`.
2. Instala `hypothesis` (`pip install hypothesis`) y escribe una prueba basada en propiedades: para cualquier número entre 0 y 99.999.999, lo rellenas con ceros a la izquierda hasta 8 cifras (`str(n).zfill(8)`) y le añades la letra que le corresponde según la tabla, `validar_dni` tiene que devolver `True`. Busca en la documentación de hypothesis cómo se usan `@given` y `st.integers`.

10. Preguntas

Contesta con tus palabras. No hace falta que sean largas, pero sí que se note que lo has entendido.

1. ¿Qué cobertura tenía el proyecto antes de que añadieras tus pruebas? ¿Qué te dice eso sobre la cobertura como medida de calidad?
2. Explica qué ventaja tiene `parametrize` frente a escribir una función por caso.
3. ¿Por qué es importante usar `tmp_path` en lugar de escribir el CSV en la carpeta del proyecto?
4. Explica con un ejemplo qué podría pasar si alguien se inscribe con un nombre que empiece por `=` y la secretaría abre el CSV con una hoja de cálculo.
5. ¿Qué fallo te costó más encontrar y qué prueba lo destapó?

11. Qué tienes que entregar

Entrega una carpeta `NN-PPS-pytest` (NN es tu número de lista) comprimida en ZIP con:

1. El proyecto entero **sin la carpeta** `.venv`.
2. Tu `tests/test_especificacion.py` y tu `tests/conftest.py`.
3. Un fichero `salida-pytest.txt` con la salida de la última ejecución con cobertura.
4. Un fichero `informe.md` con la lista de fallos encontrados (regla, dato de prueba, valor que salía, corrección aplicada) y las respuestas a las preguntas del apartado 10.

El fichero `salida-pytest.txt` lo generas así, desde la carpeta del proyecto:

```
python -m pytest --cov=inscripciones --cov-report=term-missing \
    --cov-fail-under=90 > salida-pytest.txt
```

Para crear el ZIP sin el entorno virtual:

```
cd ~/pps
zip -r NN-PPS-pytest.zip PPS-2026-11-03-pytest-LAB \
    -x "*/.venv/*" "*/__pycache__/*" "*/.pytest_cache/*"
```

Si no te ha dado tiempo a terminar, entrega lo que tengas y explica en `informe.md` en qué punto exacto te has quedado y qué estabas intentando.

12. Antes de entregar, comprueba que

1. `python -m pytest` termina sin ninguna prueba en rojo.
2. La cobertura llega al 90 % con `--cov-fail-under=90`.
3. Hay al menos una prueba con `parametrize`, una con `pytest.raises` y una con `tmp_path`.
4. Cada prueba de `test_especificacion.py` se puede relacionar con una regla (ponle el número de regla en el nombre: `test_r4_...`).
5. El ZIP no lleva `.venv`.
6. `informe.md` explica cada fallo que has corregido.

13. Un apunte de seguridad

La inyección de fórmulas en CSV no es un ejemplo de clase: la recoge OWASP como *CSV Injection* y aparece en auditorías reales, porque muchas aplicaciones exportan datos que luego alguien abre con Excel o LibreOffice. La defensa está en el momento de escribir el fichero, no en el de leerlo, y una prueba automática es la manera de asegurarte de que nadie la quita sin darse cuenta.

Los datos de la práctica son inventados. No uses DNI ni correos reales de nadie, tampoco el tuyo.

Referencias

- Documentación de pytest: https://docs.pytest.org/
- Parametrizar pruebas: https://docs.pytest.org/en/stable/how-to/parametrize.html
- Fixtures y `tmp_path`: https://docs.pytest.org/en/stable/how-to/tmp_path.html
- pytest-cov: https://pytest-cov.readthedocs.io/
- OWASP, CSV Injection: https://owasp.org/www-community/attacks/CSV_Injection